

UNIVERSIDADE DO OESTE DE SANTA CATARINA
CURSO DE SISTEMAS DE INFORMAÇÃO
DESENVOLVIMENTO PARA DISPOSITIVOS MÓVEIS

JORGE LUIZ DO NASCIMENTO

JCONDO MOBILE

Documentação técnica do desenvolvimento do aplicativo Android e da API REST

Trabalho apresentado ao componente curricular Desenvolvimento para Dispositivos Móveis, referente à Atividade Avaliativa 2, sob orientação do professor da disciplina.

CHAPECÓ
2026

SUMÁRIO

1	INTRODUÇÃO
2	VISÃO GERAL DA SOLUÇÃO
3	ATENDIMENTO AOS REQUISITOS
3.1	Requisitos funcionais
3.2	Requisitos não funcionais
3.3	Regras de negócio
3.4	Critérios de avaliação da atividade
4	A API REST
4.1	Modelo de dados
4.2	Endpoints
4.3	Autenticação e sessão
4.4	Padronização de erros
5	A REGRA DE CONFLITO DE RESERVA
6	O APLICATIVO ANDROID
6.1	Organização do código
6.2	Navegação
6.3	Integração com a API
7	EXPERIÊNCIA DO USUÁRIO
8	RECURSOS DO DISPOSITIVO
9	TESTES
10	DIFICULDADES E SOLUÇÕES
11	LIMITAÇÕES E EVOLUÇÃO
12	CONCLUSÃO
	REFERÊNCIAS

1 INTRODUÇÃO

Este documento descreve a implementação do JCondo Mobile, aplicativo Android de gestão condominial. A etapa anterior do trabalho definiu o problema, os requisitos, a plataforma e os protótipos de tela. Esta etapa transformou esse projeto em software funcionando.

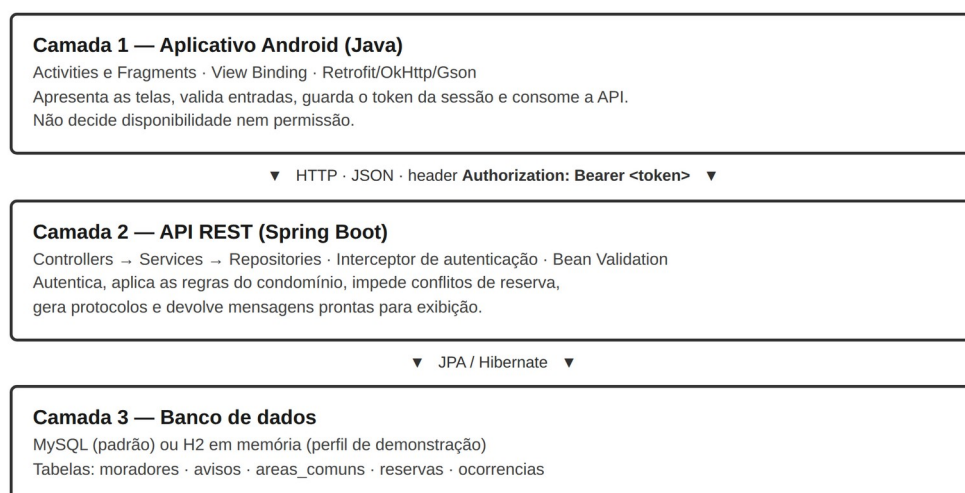
O ponto de partida do lado servidor foi o JCondo que desenvolvi no projeto integrador, uma aplicação Spring Boot com CRUD de moradores em Thymeleaf e uma API REST documentada com Swagger. Estendi esse projeto em vez de começar de novo: as telas web e o CRUD original continuam funcionando exatamente como estavam, e sobre eles acrescentei as entidades, as regras e os endpoints que o aplicativo consome.

O documento está organizado para responder, antes de tudo, a uma pergunta prática: cada requisito levantado na etapa anterior foi atendido, e onde ele está no código. Essa rastreabilidade está na seção 3. As seções seguintes entram nas decisões técnicas, no que deu trabalho e no que ficou de fora de propósito.

2 VISÃO GERAL DA SOLUÇÃO

O morador entra com e-mail e senha, consulta os comunicados da administração, reserva áreas comuns vendo os horários realmente disponíveis, registra ocorrências de manutenção e acompanha cada chamado por um número de protocolo. A arquitetura em três camadas prevista no projeto foi mantida.

Figura 1 - Arquitetura implementada



Princípio de separação: nenhuma regra de negócio existe apenas na camada 1. O aplicativo antecipa validações para dar retorno rápido, mas toda decisão que altera dados é reavaliada na camada 2.

Fonte: o autor (2026).

A decisão que sustenta o resto do trabalho é que nenhuma regra de negócio existe apenas no aplicativo. O aplicativo antecipa validações para dar retorno rápido ao morador, mas toda decisão que altera dados é reavaliada no servidor. Isso significa que um APK modificado, uma chamada feita direto pelo Postman ou dois moradores agindo ao mesmo tempo não conseguem furar as regras do condomínio.

2.1 Estrutura do repositório

```
jcondo-mobile/  
  backend/    API REST em Spring Boot 3.5 + Java 21  
  mobile/     Aplicativo Android em Java, Gradle e Retrofit  
  docs/       Documentação do projeto
```

2.2 Tecnologias utilizadas

Tabela 1 - Tecnologias e versões efetivamente usadas

Camada	Tecnologia	Versão
Aplicativo	Android SDK (compileSdk 35, minSdk 26, targetSdk 35)	API 26 a 35
Aplicativo	Java	17
Aplicativo	Gradle / Android Gradle Plugin	8.9 / 8.7.3
Aplicativo	Material Design 3	1.12.0
Aplicativo	Retrofit + Gson + OkHttp	2.11.0 / 4.12.0
Aplicativo	View Binding	nativo do AGP
API	Java	21
API	Spring Boot (Web, Data JPA, Validation)	3.5.0
API	springdoc-openapi (Swagger UI)	2.8.6
Banco	MySQL (padrão) / H2 em memória (perfil de demonstração)	8.x / 2.x
Testes	JUnit 5 + Mockito	via spring-boot-starter-test

Fonte: o autor (2026).

3 ATENDIMENTO AOS REQUISITOS

As tabelas desta seção percorrem cada requisito e cada regra definidos na etapa anterior, dizendo se foi implementado, onde está no código e como verificar. É a parte do documento que serve como checklist da entrega.

3.1 Requisitos funcionais

Tabela 2 - Rastreabilidade dos requisitos funcionais

ID	Situação	Onde está implementado	Como verificar
RF01	Atendido	AuthRestController + AutenticacaoService (API); LoginActivity (app)	Entrar com ana.souza@email.com / 123456
RF02	Atendido	SessionManager grava o token no SharedPreferences	Fechar e reabrir o app: entra direto na tela inicial
RF03	Atendido	MainActivity.sair() + POST /api/auth/logout	Perfil → Sair da conta
RF04	Atendido	PerfilRestController.resumo() + HomeFragment	A tela inicial mostra saudação, contadores e próxima reserva
RF05	Atendido	Botões atalhoReservar e atalhoOcorrencia em fragment_home	Tocar nos dois botões no rodapé da tela inicial
RF06	Atendido	AvisoRepository.findByAtivoTrueOrderByDataPublicacaoDesc	Aba Avisos: o mais recente aparece primeiro
RF07	Atendido	AvisoAdapter (etiqueta) + AvisoDetalheActivity	Tocar em um aviso para abrir o texto completo
RF08	Atendido	AreaComumRestController.listar()	Nova reserva → abrir o seletor de área
RF09	Atendido	ReservaService.agenda() + HorarioAdapter	Escolher área e data: os ocupados ficam cinza
RF10	Atendido	ReservaService.criar() + NovaReservaActivity	Selecionar horário livre e confirmar
RF11	Atendido	ReservaService.cancelar() + ReservasFragment	Aba Reservas → Cancelar reserva
RF12	Atendido	OcorrenciaService.abrir() + NovaOcorrenciaActivity	Aba Ocorrências → Nova ocorrência
RF13	Atendido	OcorrenciaService.gerarProtocolo()	Ao registrar, um diálogo mostra o protocolo
RF14	Atendido	OcorrenciaRestController.minhas() + OcorrenciaDetalheActivity	Abrir a ocorrência da Ana: mostra resposta da administração
RF15	Atendido	PerfilRestController.atualizarPerfil() + PerfilFragment	Perfil → alterar telefone → Salvar alterações
RF16	Atendido (local)	Notificacoes.notificarSeNovo()	Publicar um aviso pelo Swagger e recarregar a tela inicial
RF17	Atendido	Interruptor no PerfilFragment	Perfil → desligar o interruptor de notificação
RF18	Atendido	POST /api/avisos, restrito ao perfil ADMIN	Logar como sindico@jcondo.com e

ID	Situação	Onde está implementado	Como verificar
			publicar pelo Swagger
RF19	Não implementado	Previsto como evolução (seção 11)	-

Fonte: o autor (2026).

Dos dezenove requisitos funcionais, os quinze marcados como essenciais estão implementados. Dos quatro desejáveis, três também entraram. O único que ficou de fora é a foto na ocorrência (RF19), pela razão explicada na seção 11.

O RF16 aparece como "atendido (local)" porque a notificação é disparada pelo próprio aplicativo ao detectar um aviso novo, e não empurrada pelo servidor com o aplicativo fechado. A diferença está explicada na seção 8.

3.2 Requisitos não funcionais

Tabela 3 - Rastreabilidade dos requisitos não funcionais

ID	Situação	Como foi atendido
RNF01	Atendido	A barra inferior dá acesso às cinco áreas em um toque, e as duas ações mais comuns (reservar e abrir ocorrência) têm atalho na tela inicial.
RNF02	Atendido	Reserva e ocorrência terminam em diálogo de confirmação; ações rápidas usam Snackbar; toda operação em andamento mostra progresso e desabilita o botão.
RNF03	Atendido	A tela inicial faz uma única chamada (/api/home/resumo) em vez de quatro. Timeouts de 15s para conectar e 20s para ler.
RNF04	Atendido	RespostaApi distingue falta de rede de servidor fora do ar, e o componente EstadoUi mostra o motivo com botão de tentar novamente.
RNF05	Atendido	SenhaUtil aplica hash SHA-256 com sal. O campo senha é anotado como WRITE_ONLY: o Jackson aceita recebê-lo, mas nunca o devolve.
RNF06	Atendido	Os endpoints do morador não recebem identificador por parâmetro. O dono é extraído do token pelo AuthInterceptor.
RNF07	Atendido	Layouts com pesos e medidas relativas, listas em RecyclerView e telas roláveis.
RNF08	Atendido	Estilos com android:minHeight de 48dp nos botões e alvos de toque; paleta verificada para contraste acima de 4,5:1.
RNF09	Atendido	A prioridade do aviso traz o texto ("Urgente", "Importante") junto da cor; o horário ocupado exibe a palavra "Ocupado".
RNF10	Atendido	Backend em controller/service/repository; aplicativo separado em api, session, ui e util, com uma pasta por área do MVP.
RNF11	Atendido	minSdk 26 declarado no build.gradle do módulo app.
RNF12	Atendido	Persistência em MySQL via Spring Data JPA; o SharedPreferences guarda apenas token e preferências.
RNF13	Atendido	O manifesto declara apenas INTERNET, ACCESS_NETWORK_STATE e POST_NOTIFICATIONS. Nenhum sensor ou serviço em segundo plano.

Fonte: o autor (2026).

3.3 Regras de negócio

Tabela 4 - Rastreabilidade das regras de negócio

ID	Onde a regra é aplicada	Teste automatizado
RN01	ReservaRepository.contarConflitos(), consultada por ReservaService.criar()	deveRecusarReservaQuandoHorarioJaEstaOcupado
RN02	A condição de sobreposição usa comparação estrita (< e >), não inclusiva	agendaDeveMarcarComoIndisponivelApensasAFaixaOcupada
RN03	ReservaService.criar(): data anterior a hoje e horário já passado	deveRecusarReservaEmDataPassada
RN04	ReservaService.criar(): compara com horarioAbertura e horarioFechamento da área	deveRecusarHorarioForaDoFuncionamentoDaArea
RN05	ReservaService.criar(): limite de 90 dias	coberto pela mesma validação de data

ID	Onde a regra é aplicada	Teste automatizado
RN06	ReservaService.cancelar(): confere dono e se a reserva já ocorreu	naoDeveCancelarReservaDeOutroMorador e deveCancelarReservaFuturaDoProprioMorador
RN07	OcorrenciaService.gerarProtocolo(), com unicidade também no banco	garantida por constraint UNIQUE na coluna protocolo
RN08	PerfilRestController.atualizarPerfil() só altera telefone, e-mail e senha	verificável pelo Swagger: enviar bloco no corpo não altera nada
RN09	AvisoRestController.publicar() verifica UsuarioLogado.isAdmin()	verificável logando como morador e tentando publicar

Fonte: o autor (2026).

3.4 Critérios de avaliação da atividade

Além dos requisitos do projeto, a atividade define quatro critérios de avaliação. A tabela relaciona cada um ao que foi entregue.

Tabela 5 - Atendimento aos critérios da atividade

Critério	O que foi entregue
Funcionamento e desempenho conforme os requisitos do projeto	As seis telas do MVP implementadas e funcionando contra a API. Quinze de quinze requisitos essenciais atendidos, mais três dos quatro desejáveis. A tela inicial reduzida a uma única chamada de rede.
Integração com APIs e comunicação com servidores	Dezesseis endpoints REST consumidos via Retrofit, com autenticação por token anexada por interceptor, tratamento de erro padronizado e a regra de conflito resolvida no servidor com resposta HTTP 409 específica.
Qualidade do design e criatividade	Material Design 3 com identidade própria definida na etapa anterior, estados de carregamento, vazio e erro em todas as listas, etiquetas que não dependem só de cor e grade de horários que mostra o ocupado em vez de escondê-lo.
Clareza e organização na apresentação	Esta documentação, o guia de execução passo a passo, o roteiro de apresentação e o README do repositório, além da API documentada em Swagger e em arquivo próprio.

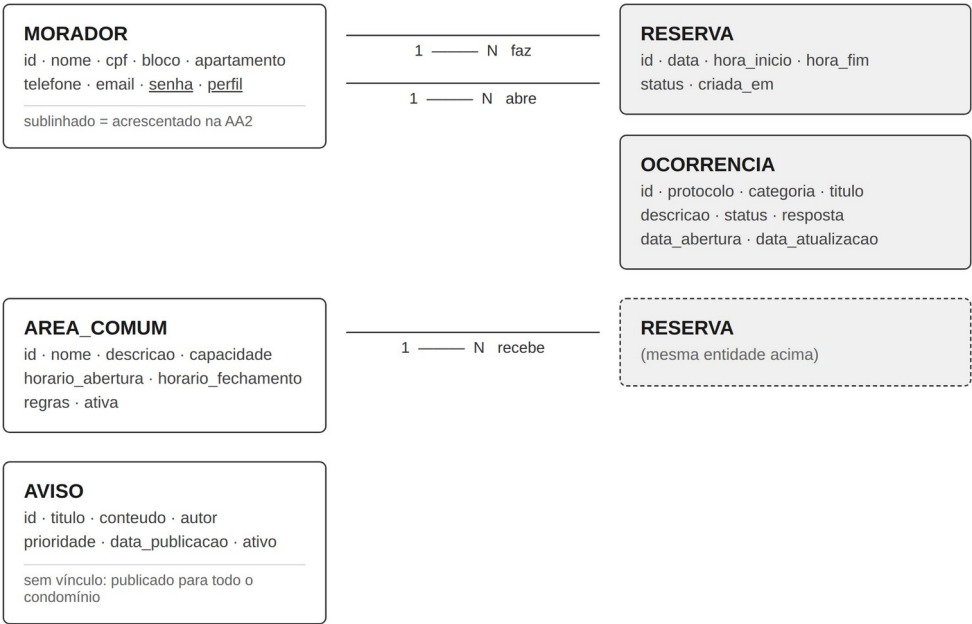
Fonte: o autor (2026).

4 A API REST

O backend da etapa anterior tinha uma entidade e cinco endpoints. Para atender ao MVP, acrescentei quatro entidades, a autenticação por token e dezesseis endpoints.

4.1 Modelo de dados

Figura 2 - Modelo de entidades implementado



Fonte: o autor (2026).

Optei por aproveitar a entidade Morador como usuário do aplicativo, em vez de criar uma entidade Usuario separada. No condomínio, todo usuário do app é um morador cadastrado, e uma segunda tabela só para credenciais criaria a obrigação de manter dois cadastros sincronizados sem trazer benefício.

4.2 Endpoints

Tabela 6 - Endpoints da API

Método e rota	Função
POST /api/auth/login	Autentica e devolve o token de sessão com os dados do morador.
POST /api/auth/logout	Invalida o token atual.
GET /api/home/resumo	Saudação, contadores, próxima reserva e avisos recentes em uma chamada.
GET /api/perfil	Dados do morador autenticado.

Método e rota	Função
PUT /api/perfil	Atualiza telefone, e-mail e senha do próprio cadastro.
GET /api/aviso	Lista os avisos ativos, do mais recente para o mais antigo.
GET /api/aviso/{id}	Consulta um aviso.
POST /api/aviso	Publica um aviso (somente perfil ADMIN).
GET /api/areas	Lista as áreas comuns disponíveis para reserva.
GET /api/areas/{id}/horarios	Agenda da área em uma data, em faixas de 2 horas.
GET /api/reservas/minhas	Reservas do morador autenticado.
POST /api/reservas	Cria a reserva. Devolve 409 quando o horário já está ocupado.
DELETE /api/reservas/{id}	Cancela uma reserva própria que ainda não ocorreu.
GET /api/ocorrencias/minhas	Ocorrências do morador autenticado.
GET /api/ocorrencias/{id}	Consulta uma ocorrência própria.
GET /api/ocorrencias/categorias	Categorias aceitas, para preencher o seletor do formulário.
POST /api/ocorrencias	Abre uma ocorrência e devolve o protocolo gerado.
GET, POST, PUT, DELETE /api/moradores	CRUD herdado da etapa anterior, agora protegido por token.

Fonte: o autor (2026).

Uma decisão de segurança percorre todos os endpoints do morador: nenhum deles recebe o identificador do usuário por parâmetro. As rotas são /api/reservas/minhas e /api/perfil, e não /api/reservas?moradorId=3. O dono da requisição sai sempre do token, o que impede consultar ou alterar dados de outro morador trocando um número na URL.

4.3 Autenticação e sessão

O login recebe e-mail e senha e devolve um token opaco, válido por doze horas. Um interceptor do Spring MVC valida esse token em todas as rotas /api/**, com exceção do próprio login, e disponibiliza o morador autenticado aos controllers. No aplicativo, o token fica no SharedPreferences e é anexado automaticamente pelo interceptor do OkHttp, de modo que nenhuma tela precisa saber que ele existe.

As senhas passam por hash SHA-256 com um sal fixo da aplicação e nunca são gravadas em texto puro. A comparação usa MessageDigest.isEqual, que percorre todos os bytes independentemente de onde ocorra a primeira diferença, para não vaziar informação pelo tempo de resposta. O login também responde a mesma mensagem para e-mail inexistente e para senha errada, porque mensagens distintas revelariam quais e-mails estão cadastrados.

4.4 Padronização de erros

Todo erro da API sai no mesmo formato, com um campo de mensagem já escrito em português e pronto para exibição:

```
{
  "status": 409,
  "erro": "conflito_reserva",
  "mensagem": "O horário das 18:00 às 20:00 já está reservado
               em Salão de Festas. Escolha outro horário.",
  "campos": null,
  "momento": "2026-09-05T14:22:10.113"
}
```

Isso resolve um problema real de aplicativo que consome API: sem essa padronização, cada tela precisaria traduzir códigos HTTP por conta própria, e as mensagens acabariam divergindo entre telas. Com o formato único, o aplicativo lê um campo só.

5 A REGRA DE CONFLITO DE RESERVA

Entre todos os requisitos, o que exige mais cuidado é o RN01: impedir que duas reservas ocupem o mesmo horário na mesma área. É o único ponto do sistema em que dois usuários podem, de boa-fé, tentar fazer a mesma coisa ao mesmo tempo.

A verificação acontece no banco, com uma consulta que conta as reservas confirmadas que se sobrepõem à faixa pedida:

```
select count(r) from Reserva r
where r.area.id = :areaId
and r.data = :data
and r.status = 'CONFIRMADA'
and r.horaInicio < :horaFim
and r.horaFim > :horaInicio
```

A condição de sobreposição é a comparação cruzada: duas faixas conflitam quando o início de uma é anterior ao fim da outra e o fim é posterior ao início dela. Escrever assim resolve o caso das reservas encostadas exigido pelo RN02. Uma reserva das 14h às 16h e outra das 16h às 18h não conflitam, porque o fim de uma coincide com o início da outra sem invadi-la.

O teste que mudou o código

A primeira versão da consulta usava comparações inclusivas ($\leq$ e $\geq$) e recusava a reserva das 16h às 18h quando já existia uma das 14h às 16h. Escrevi o teste `agendaDeveMarcarComoIndisponivelApenasAFaixaOcupada` antes de perceber o erro, e foi ele que apontou o problema. Hoje esse teste é o que impede o comportamento de se perder de novo em uma alteração futura.

Antes da checagem de conflito, o serviço aplica outras seis validações: a área existe e está ativa, o término é posterior ao início, a data não está no passado, o horário ainda não passou se for hoje, a faixa cabe dentro do funcionamento da área e a antecedência não passa de noventa dias.

Figura 3 - Sequência quando dois moradores tentam o mesmo horário

Dois moradores tentam o mesmo horário

- 1 Ambos abrem **Nova reserva** e veem a mesma agenda: 18:00–20:00 livre.
- 2 Morador A confirma → **POST /api/reservas**
- 3 API roda as validações e a consulta de sobreposição no banco: `horaInicio < :horaFim AND horaFim > :horaInicio` → **0 conflitos**
- 4 Grava a reserva · responde **201 Created** · o app de A mostra a confirmação
- 5 Morador B confirma o **mesmo horário** → **POST /api/reservas**
- 6 A mesma consulta agora encontra **1 conflito**
- 7 API responde **409 Conflict** com a mensagem explicando qual horário está ocupado
- 8 O app de B exibe a mensagem, limpa a seleção e **recarrega a agenda**, que agora mostra 18:00–20:00 como ocupado

Por que no servidor: se a verificação estivesse apenas no aplicativo, os dois teriam gravado, porque cada um consultou a agenda antes de o outro confirmar. A decisão precisa acontecer no mesmo lugar onde o dado é gravado.

Fonte: o autor (2026).

No aplicativo, a tela de nova reserva consulta a agenda do dia e desabilita as faixas ocupadas. Isso é uma facilidade visual, não a regra. Se dois moradores abrirem a mesma tela e confirmarem o mesmo horário no mesmo instante, o servidor grava a primeira e devolve 409 para a segunda. O aplicativo trata esse código de forma específica: mostra a mensagem, limpa a seleção e recarrega a agenda, deixando a tela coerente com o servidor sem que o morador precise sair e voltar.

6 O APLICATIVO ANDROID

6.1 Organização do código

```
com.jcondo.mobile
  api/      contrato Retrofit, cliente HTTP e tradução de erros
  model/    objetos que representam o JSON da API
  session/  SessionManager: token, morador e endereço da API
  ui/       uma pasta por área do MVP
    home/   tela inicial
    avisos/ lista e leitura de comunicados
    reservas/ minhas reservas, nova reserva e grade de horários
    ocorrencias/ lista, abertura e acompanhamento
    perfil/  dados do morador e preferências
    util/    datas, notificações, estados de tela e atalhos de UI
```

A divisão por área do MVP, em vez de por tipo de classe, foi pensada para a manutenção. Quem for mexer em reservas encontra tudo o que precisa em uma pasta só: o fragmento da lista, o adaptador, a tela de criação e o adaptador da grade de horários.

6.2 Navegação

Figura 4 - Fluxo de navegação implementado



Fonte: o autor (2026).

A navegação usa uma Activity principal com barra inferior e cinco fragmentos, conforme o protótipo. Os fragmentos já criados são ocultados e reexibidos em vez de

recriados, então voltar para uma aba preserva a posição da lista e não dispara uma nova chamada à API a cada toque.

Três telas de lista, avisos, reservas e ocorrências, compartilham o mesmo arquivo de layout. A estrutura delas é idêntica: título, lista, estado e ação principal. Reaproveitar o layout garante que se comportem da mesma forma, que é o princípio de consistência definido na etapa anterior.

6.3 Integração com a API

A comunicação usa Retrofit, que descreve os endpoints como métodos de uma interface Java. O ponto mais importante da configuração é o interceptor do OkHttp: ele anexa o header Authorization a toda requisição que não seja o login, lendo o token do SessionManager.

Isso tem duas consequências práticas. Primeiro, nenhuma das dezesseis chamadas espalhadas pelas telas precisa lembrar de enviar o token, então não existe a possibilidade de esquecer em uma delas. Segundo, trocar o mecanismo de autenticação no futuro, por exemplo para JWT assinado, mexe em um arquivo só.

O endereço da API é configurável em tempo de execução, na própria tela de login. O padrão é 10.0.2.2, que é como o emulador enxerga o localhost da máquina, mas o mesmo APK roda em um celular físico apontando para o IP do computador na rede Wi-Fi, sem recompilar.

6.3.1 Tradução de erros

Um aplicativo que consome API precisa responder três perguntas quando algo falha: o que aconteceu, de quem é a falha e o que o usuário pode fazer. Sem tratamento, uma queda de Wi-Fi apareceria na tela como `java.net.ConnectException` e um horário ocupado como HTTP 409.

Tabela 7 - Tradução das falhas em mensagens

Situação	O que o morador lê
Sem rede (IOException)	Sem conexão com a internet. Verifique a rede e tente de novo.
API fora do ar ou endereço errado	Não foi possível falar com o servidor do condomínio. Confira o endereço da API e se ela está no ar.
HTTP 409 ao reservar	O horário das 18:00 às 20:00 já está reservado em Salão de Festas. Escolha outro horário.
HTTP 401	Sua sessão expirou. Entre novamente.
HTTP 400 de validação	A primeira mensagem de campo devolvida pelo servidor, marcada no próprio campo do formulário.

Situação	O que o morador lê
HTTP 500	Erro inesperado no servidor. Tente novamente em instantes. (sem stack trace)

Fonte: o autor (2026).

7 EXPERIÊNCIA DO USUÁRIO

A etapa anterior listou seis diretrizes de interface. A tabela mostra como cada uma se materializou no código.

Tabela 8 - Diretrizes de UX e sua implementação

Diretriz	Implementação
Hierarquia visual	A tela inicial apresenta, nesta ordem: quem é o morador, os números do condomínio, o compromisso mais próximo, os avisos recentes e os atalhos.
Consistência	Uma escala tipográfica curta e um conjunto único de estilos para cartões, botões e etiquetas atravessam todas as telas. As três listas usam o mesmo layout.
Feedback	Reserva confirmada e ocorrência registrada abrem um diálogo com o resultado. Ações rápidas usam Snackbar. Toda operação em andamento mostra progresso e desabilita o botão, impedindo envio duplicado.
Acessibilidade	Alvos de toque de no mínimo 48dp, contraste acima de 4,5:1 e informação nunca transmitida apenas por cor. A grade de horários anuncia "disponível" ou "ocupado" também para o leitor de tela.
Responsividade	Layouts com pesos e limites relativos, telas roláveis e listas em RecyclerView, adaptando-se de telas pequenas a tablets.
Redução de etapas	A tela inicial leva a uma nova reserva ou a uma nova ocorrência em um toque. A reserva completa exige três decisões e nada mais.

Fonte: o autor (2026).

7.1 O problema da tela em branco

Um cuidado que não estava previsto na etapa anterior apareceu durante a implementação. Toda lista alimentada por API tem quatro estados possíveis: carregando, com conteúdo, vazia e com erro. Tratar apenas o segundo produz aquele momento em que a tela fica branca e o usuário não sabe se o aplicativo travou, se a internet caiu ou se realmente não há nada para ver.

Criei um componente único que cobre os outros três estados, reutilizado pelas telas de avisos, reservas, ocorrências e início. O morador sempre vê ou um indicador de progresso, ou uma explicação de por que a lista está vazia com o caminho para preenchê-la, ou o motivo da falha com um botão para tentar de novo. Como o componente é o mesmo, as quatro telas se comportam igual nessas situações.

8 RECURSOS DO DISPOSITIVO

A etapa anterior definiu um uso contido dos recursos do aparelho, para preservar bateria e evitar coleta desnecessária de dados. Essa decisão foi mantida: o

aplicativo declara apenas as permissões de internet, estado da rede e notificação. Não pede GPS, câmera nem acesso a arquivos.

Tabela 9 - Uso de recursos do aparelho na implementação

Recurso	Uso no aplicativo
Rede	Única fonte de dados. Timeouts de 15s para conectar e 20s para ler, para avisar rápido em rede ruim em vez de deixar a tela girando.
Notificações	Canal próprio criado na inicialização. O aplicativo compara o aviso mais recente com o último já visto e notifica só quando há algo novo. A permissão é pedida em tempo de execução, como exige o Android 13.
Armazenamento local	SharedPreferences com token, dados básicos do morador, endereço da API e preferência de notificação. A senha não é guardada, e o arquivo fica excluído do backup do aparelho.
GPS e câmera	Não utilizados, conforme decidido no projeto.

Fonte: o autor (2026).

Sobre as notificações, cabe uma observação honesta: o que está implementado é notificação local, disparada pelo próprio aplicativo ao detectar um aviso novo durante o carregamento da tela inicial. O push real, que chega com o aplicativo fechado, exigiria Firebase Cloud Messaging, ou seja, um projeto no Firebase e um serviço de envio no backend. Ficou registrado como evolução.

9 TESTES

Os testes automatizados se concentram onde um erro seria mais caro: na regra de reserva. Foram escritos com JUnit 5 e Mockito, com o repositório simulado, de modo que rodam sem banco de dados.

Tabela 10 - Cobertura dos testes automatizados

Classe de teste	O que verifica
ReservaServiceTest (9 testes)	Criação de reserva sem conflito; recusa quando o horário está ocupado; recusa de data passada, de horário fora do funcionamento e de término anterior ao início; geração correta da agenda, incluindo o caso das faixas encostadas; impedimento de cancelar reserva de outro morador; cancelamento de reserva futura própria; contagem de reservas ativas.
SenhaUtilTest (3 testes)	O hash não devolve a senha original, é determinístico, e a verificação recusa senha errada, vazia e nula.
TokenServiceTest (3 testes)	Token emitido aponta para o morador correto, cada login gera token distinto, e token inexistente ou revogado não autentica.
MoradorServiceTest (5 testes)	Operações de CRUD da etapa anterior, mantidas sem alteração.

Fonte: o autor (2026).

Para executar:

```
cd backend
./mvnw test
```

10 DIFICULDADES E SOLUÇÕES

Tabela 11 - Problemas encontrados durante o desenvolvimento

Dificuldade	Solução adotada
O emulador não enxerga o localhost do computador.	Uso do endereço 10.0.2.2, que o emulador redireciona para a máquina hospedeira, e um campo de configuração de servidor na tela de login para testar em celular físico sem recompilar.
O Android bloqueia tráfego HTTP sem TLS desde a versão 9.	Arquivo <code>network_security_config</code> liberando cleartext apenas para faixas de endereço privadas, mantendo o bloqueio para o restante da internet.
Adicionar senha ao morador quebraria o formulário web da etapa anterior, que não envia esse campo e gravaria nulo por cima do hash.	A camada de serviço detecta se o valor recebido é senha nova em texto puro ou hash já gravado, e preserva o existente quando o campo vem vazio.
Objetos JPA com relacionamentos geram JSON aninhado e recursivo, pesado para o aplicativo.	Uso de DTOs em formato de record, achatando os dados e já entregando datas formatadas em português, para a tela não fazer conversão.
A tela inicial precisava de dados de quatro origens e montava aos	Criação do endpoint <code>/api/home/resumo</code> , que devolve tudo em uma chamada. Menos tráfego, menos bateria e a tela carrega de uma

Dificuldade	Solução adotada
pedaços.	vez.
Avaliar o trabalho exigiria instalar e configurar o MySQL.	Perfil h2 com banco em memória e carga automática de dados de exemplo: um comando sobe a API completa e populada.
A primeira consulta de conflito recusava reservas encostadas.	Troca das comparações inclusivas por estritas, com teste automatizado cobrindo o caso.

Fonte: o autor (2026).

11 LIMITAÇÕES E EVOLUÇÃO

Algumas decisões foram tomadas de forma consciente para manter o escopo compatível com a atividade. Registrá-las é mais útil do que omiti-las.

- As senhas usam SHA-256 com sal fixo. Isso impede o armazenamento em texto puro, mas um algoritmo rápido é vulnerável a força bruta em GPU. Em produção o correto seria BCrypt ou Argon2, com sal por usuário, o que exigiria trazer o Spring Security para o projeto.
- As sessões ficam em memória no servidor. Reiniciar a aplicação derruba os logins. A evolução natural é um JWT assinado ou uma tabela de sessões.
- A comunicação é HTTP na rede local. Em produção a API precisaria estar atrás de HTTPS.
- As notificações são locais, e não push via Firebase.
- A foto na ocorrência (RF19) não foi implementada. Ela exigiria upload de arquivo, armazenamento no servidor e permissão de câmera, e preferi consolidar os requisitos essenciais.
- A administração publica avisos pela API, mas ainda não há tela administrativa no aplicativo. O CRUD web da etapa anterior continua sendo o caminho para a gestão de moradores.

11.1 Preparação para as próximas versões

O projeto foi organizado para que a continuação não exija reescrita. Três decisões contribuem diretamente para isso.

A primeira é a lógica de negócio estar no servidor. Uma futura versão iOS, ou uma tela web, consome a mesma API e herda as mesmas regras, sem duplicação.

A segunda é a separação em camadas dentro de cada projeto. No backend, acrescentar uma funcionalidade significa criar entidade, repositório, serviço, DTO e controller, sempre nos mesmos cinco lugares. No aplicativo, significa criar uma pasta em ui, acrescentar os métodos na interface do Retrofit e incluir um item na navegação.

A terceira são os pontos de extensão já preparados: o componente de estados de tela, o tratador único de erros, o layout de lista compartilhado e o interceptor de autenticação. Qualquer tela nova nasce com esses quatro comportamentos prontos. O repositório traz um guia de evolução com o passo a passo dessa adição.

11.2 Evoluções previstas

Mantendo o que foi planejado na etapa anterior: pagamento de taxas e segunda via de boletos, documentos e atas, assembleias e votações digitais, conversa com a administração, integração com a portaria, QR Code para visitantes e painel administrativo.

12 CONCLUSÃO

O JCondo Mobile implementa o MVP definido na etapa anterior: autenticação com controle de acesso, tela inicial, avisos, reservas de áreas comuns com validação de conflito, registro e acompanhamento de ocorrências e perfil do morador. As telas seguem os wireframes e a identidade visual projetados, e a arquitetura em três camadas foi mantida do desenho à implementação.

O planejamento da primeira etapa cumpriu bem seu papel. Requisitos, entidades e fluxos definidos previamente encurtaram as decisões durante o desenvolvimento, e a maior parte do esforço foi de implementação, não de descobrir o que construir. Os pontos que exigiram decisão nova apareceram onde o projeto não tinha como antecipar: o comportamento diante de falhas de rede, o tratamento dos estados vazios das listas e a forma exata de calcular a sobreposição de horários.

A integração com a API foi o eixo do trabalho. O aprendizado mais concreto não foi como chamar um endpoint, e sim onde colocar cada responsabilidade: o servidor decide, o aplicativo apresenta e antecipa. É essa divisão que mantém a regra do condomínio válida mesmo quando dois moradores agem ao mesmo tempo, e é ela que permite ao aplicativo evoluir sem colocar em risco a integridade dos dados.

REFERÊNCIAS

ANDROID DEVELOPERS. Guia de arquitetura de apps. Disponível em: <https://developer.android.com/topic/architecture>. Acesso em: set. 2026.

ANDROID DEVELOPERS. Network security configuration. Disponível em: <https://developer.android.com/privacy-and-security/security-config>. Acesso em: set. 2026.

ANDROID DEVELOPERS. Notification runtime permission. Disponível em: <https://developer.android.com/develop/ui/views/notifications/notification-permission>. Acesso em: set. 2026.

FOWLER, Martin. Padrões de arquitetura de aplicações corporativas. São Paulo: Novatec, 2006.

GOOGLE. Material Design 3: guidelines de componentes e acessibilidade. Disponível em: <https://m3.material.io>. Acesso em: set. 2026.

PRESSMAN, Roger S. Engenharia de software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SPRING. Spring Boot reference documentation. Disponível em: <https://docs.spring.io/spring-boot/docs/current/reference/html>. Acesso em: set. 2026.

SQUARE. Retrofit: a type-safe HTTP client for Android and Java. Disponível em: <https://square.github.io/retrofit>. Acesso em: set. 2026.

UNOESC. Desenvolvimento para dispositivos móveis. v. 2. Porto Alegre: SAGAH. Material didático da disciplina.

W3C. Web Content Accessibility Guidelines (WCAG) 2.1. Disponível em: <https://www.w3.org/TR/WCAG21>. Acesso em: set. 2026.